`IWebHostEnvironment` is already set by the time the `WebApplication` instance is created. `EnvironmentName` is typically set externally by using an environment variable when your application starts.

Listing 3.4 added only a single piece of middleware to the pipeline, but `WebApplication` *automatically* adds more middleware, including two of the most important and substantial pieces of middleware in the pipeline: the *routing* middleware and the *endpoint* middleware. The routing middleware is added automatically to the *start* of the pipeline, before any of the additional middleware added in Program.cs (so before the `HttpLoggingMiddleware`). The endpoint middleware is added to the *end* of the pipeline, after all the other middleware added in Program.cs.

NOTE `WebApplication` adds several more pieces of middleware to the pipeline by default. *It* automatically adds error-handling middleware when you're running in the development environment, for example. I discuss some of this autoadded middleware in detail in chapter 4.

Together, this pair of middleware is responsible for interpreting the request to determine which endpoint to invoke, for reading parameters from the request, and for generating the final response. For each request, the *routing* middleware uses the request's URL to determine which endpoint to invoke. Then the rest of the middleware pipeline executes until the request reaches the endpoint middleware, at which point the endpoint middleware executes the endpoint to generate the final response.

The routing and endpoint middleware work in tandem, using the set of endpoints defined for your application. In listing 3.4 we defined two endpoints:

```
app.MapGet("/", () => "Hello World!");
app.MapGet("/person", () => new Person("Andrew", "Lock"));
```

You've already seen the default `"Hello World!"` endpoint. When you send a `GET` request to `/`, the routing middleware selects the `"Hello World!"`